

UNITÉ 1 · SEMAINE 1

Introduction aux systèmes d'exploitation

📅 Semaine 1 · dates à confirmer </> Laboratoire TD0 (corrigé + défi IA) ~ 4 animations

LABORATOIRE TD0 · CORRIGÉ

Mettre en place l'environnement de programmation système

Avant de programmer au contact du système d'exploitation, il vous faut quatre choses : savoir lire le manuel, un squelette de programme, une petite bibliothèque d'utilitaires et un `Makefile` qui compile tout en transformant les avertissements en erreurs.

</> Comment utiliser les laboratoires corrigés

Chaque laboratoire est fourni avec sa solution complète. Le code que vous voyez ici est exactement celui du dossier `tds/` du cours, compilé avec `-Wall -Wextra -Werror` et testé par `tds/tests/run.sh` ; les sorties proviennent d'une exécution réelle. Votre travail : compiler, exécuter, modifier quelque chose, prédire ce qui va se passer et le vérifier. **Vous compilez toujours avec `gcc`**, avec les avertissements transformés en erreurs : voici la ligne pour chaque programme (le `Makefile` utilise exactement la même pour tous les compiler d'un coup).

Bash

```
cd tds/td0
gcc -std=gnu11 -O2 -Wall -Wextra -Werror -D_GNU_SOURCE -pthread -I../common
errors.c ../common/utils.c -o errors -lm
./errors

make -C tds          # the same gcc line for every program of every lab
(make prints each one)
tds/tests/run.sh td0 # tests one lab and prepares its working folder
tds/tests/tmp/td0
```

**Devoir : rapport du TP**

Rendez ce TP avec un court rapport en LaTeX (le strict nécessaire). Ce qu'il doit couvrir, le modèle, un exemple et le guide de Prism sont dans l'onglet Devoir. IA générative autorisée, avec déclaration.

En option: le terminal dans le cours

Vous pouvez travailler sur les laboratoires avec un **vrai terminal de votre machine** dans cette page: chaque sortie enregistrée qui peut être reproduite a un bouton ► **Exécuter** qui lance la même commande sur votre machine, et le bouton ► **_ Terminal** (en bas à droite, sur les pages de laboratoire) ouvre un terminal où vous tapez ce que vous voulez. Un navigateur ne peut pas lancer de processus tout seul: un petit programme du cours sert donc de pont.

Bash

```
make -C tds && tds/tests/run.sh    # once: builds every lab and prepares
tds/tests/tmp/td0 ... td8
python3 tds/terminal.py           # leave this terminal open while you work
```

Le pont affiche un **lien** (ouvrez-le et le cours se connecte tout seul) et une **clé** comme `8765.1a2b...`, que vous pouvez aussi coller dans le panneau. La clé change à chaque démarrage. **Ctrl+C** le ferme, avec tous ses terminaux.

- **Toujours avec gcc** : ► **Exécuter** tape, une commande par ligne, le `cd` vers le dossier du laboratoire, un `gcc` par programme de la commande (l'exécutable va dans le dossier de travail, `tds/tests/tmp/td3`) et la commande de la sortie enregistrée, qui s'exécute là, à côté des fichiers de test. Si vous modifiez le code et appuyez de nouveau, vous voyez l'effet; si `gcc` signale une erreur, rien ne s'exécute. ⚙ **Compiler avec gcc**, sur le code de chaque exercice, se contente de compiler (avec toutes ses variantes `-D...`). Si le laboratoire n'a pas encore de dossier de travail, il le prépare d'abord avec `tests/run.sh`; **Préparer TDn** fait seulement cela. Comparez votre sortie avec celle enregistrée: PID, adresses et temps changent, et expliquer pourquoi fait partie du laboratoire.
- **Fenêtres flottantes**: chaque terminal flotte au-dessus de la page pendant que vous lisez l'énoncé, le manuel ou les animations. Déplacez-le en faisant glisser sa barre (ou avec les flèches quand la barre a le focus), redimensionnez-le depuis le coin et réduisez-le avec `-`; il se souvient de l'endroit où vous l'avez laissé. `+` en ouvre un autre (jusqu'à 3), pratique pour un serveur et son client (TD2, TD7); ► **Exécuter** tape dans le dernier utilisé. 🗒 le place dans une fenêtre séparée, toujours au premier plan, même devant d'autres applications comme le PDF du laboratoire (Firefox et Chrome récents); la fermer le ramène dans la page.
- **Ubuntu**: lancez-le dans le Terminal (`Ctrl+Alt+T`), dans le dossier du cours. Le Firefox fourni avec Ubuntu et Chrome fonctionnent tous les deux. **Windows**: lancez-le dans WSL 2 (le terminal Ubuntu), là où vous compilez les laboratoires; le navigateur de Windows atteint le `127.0.0.1` de WSL car WSL redirige `localhost` par défaut.
- **Aussi dans le navigateur**: le pont sert le hub à l'adresse `http://127.0.0.1:8765/` (le second lien qu'il affiche, avec la clé `aqui...`). Pratique sous Windows avec WSL: ouvrez-le dans le navigateur de Windows.

- **Sans Ubuntu: GitHub Codespaces.** Dans le dépôt du cours, `github.com/andresrubiop/fso-uti`, cliquez sur *Code* → *Codespaces* : vous obtenez un Ubuntu 26.04 dans le cloud, avec les TP déjà compilés et un terminal dans le navigateur. Lancez-y `python3 tds/terminal.py` et ouvrez le lien `https://...app.github.dev/?terminal=aqui...` qu'il affiche: c'est le hub avec son terminal connecté au Codespace. Le pont n'accepte que l'adresse privée de ce Codespace; laissez le port 8765 privé. Il utilise le quota gratuit de votre compte GitHub: arrêtez le codespace quand vous avez fini. Les mesures (mémoire, temps, Landlock du TD8) peuvent différer de celles d'un Ubuntu installé.
- **Sécurité:** un shell à portée du navigateur est délicat; le pont n'écoute donc que sur `127.0.0.1` (jamais sur le réseau), exige la clé de la session, refuse les pages d'autres sites, ouvre au plus 3 terminaux et ferme chacun quand vous fermez l'onglet. C'est *votre* shell, avec vos permissions: ne collez pas de commandes que vous ne comprenez pas.

Sans le pont, rien ne change: copiez les commandes dans votre terminal. Les sorties qui nécessitent un serveur séparé (TD7) ou le démon `agentd` (TD8) n'ont pas de bouton; leurs étapes sont dans l'énoncé.

EXERCICES

- | | | | |
|---|---|---|---------------------------------------|
| 1 | Pages et sections du manuel | 4 | Compiler avec <code>make</code> |
| 2 | Le squelette de programme | 5 | Explorez votre système d'exploitation |
| 3 | La bibliothèque <code>libutils</code> et <code>exit_if</code> | | |

TD0 · 1

● Corrigé

Pages et sections du manuel

Énoncé. Dans ce cours, nous écrivons `cmd(k)` pour dire que `cmd` est documenté dans la section *k* du manuel. Utilisez `man -k` pour chercher par mot-clé et répondez: dans quelle section se trouve le `printf` de la bibliothèque C? Quelles sont les autres pages qui apparaissent? Quelle est la différence entre les sections `3` et `3p`?

Solution



Bash

```
man -k printf | grep ^printf      # keyword search, keep the names that start
with printf
man 3 printf                      # the C library function
man 1 printf                      # the shell command with the same name
man 3p printf                    # the POSIX standard text (package
manpages-posix-dev)
```

SECTION	SOMMAIRE	EXEMPLE
1	Programmes et commandes du shell	ls(1), printf(1)
2	Appels système (entrées dans le noyau)	write(2), fork(2)
3	Fonctions de bibliothèque (glibc sous Linux)	printf(3), perror(3)
3p	La même fonction selon la norme POSIX: ce qui doit être vrai sur <i>n'importe quel</i> Unix	printf(3p)
5 · 7	Formats de fichiers · vues d'ensemble	passwd(5), signal(7)

Réponse. Le `printf` du C est dans la section 3; `printf(1)` est la commande du shell. La section 3 décrit l'implémentation dont vous disposez (glibc, avec ses extensions); la 3p décrit le contrat portable. Quand vous écrivez pour plusieurs systèmes, fiez-vous à la 3p; quand vous déboguez sous Linux, lisez la 3.

TD0 · 2

● Corrigé

Le squelette de programme

Énoncé. Chaque exercice part d'un programme vide qui inclut les en-têtes habituels et `utils.h`. Copiez-le sous un nouveau nom pour commencer chaque exercice.

Solution



C tds/td0/skeleton.c

```
/*
 * TD0 - skeleton.c: the empty program every exercise starts from.
 * Copy it with a new name (cp td0/skeleton.c td1/echo.c) and build with
 * `make`.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    (void)argc;    /* silences -Wunused-parameter until you use them */
    (void)argv;
    return EXIT_SUCCESS;
}
```

Les lignes `(void)argc;` existent parce que nous compilons avec `-Wextra`, qui avertit des paramètres inutilisés, et avec `-Werror`, qui transforme cet avertissement en erreur. Dès que le programme utilise `argc` et `argv`, supprimez-les.

TD0 · 3

● Corrigé · testé

La bibliothèque `libutils` et `exit_if`

Énoncé. Lisez `utils.h`. Que fait `exit_if` et pourquoi utilise-t-il `errno` ? Écrivez un programme qui ouvre un fichier et utilise `exit_if` pour signaler l'erreur s'il n'existe pas.

Solution

C tds/common/utils.h

```
/*
 * utils.h - small helper library shared by every lab (libutils.a).
 * Fundamentals of Operating Systems - Universidad Tecnológica Indoamerica.
 * Based on the PG204 labs (ENSEIRB-MatMeca, F. Morandat / M. Faverge).
 */
```

```

#ifndef FSO_UTILS_H
#define FSO_UTILS_H

#include <signal.h>      /* sigset_t */
#include <stddef.h>      /* size_t */
#include <sys/types.h>   /* ssize_t, pid_t */

/*
 * Exit with status 1 if `condition` is true.
 * The message is `prefix: <strerror(errno)>` when errno is set, `prefix`
 otherwise.
 * errno is only meaningful right after a failed call, so use it right after
 the call you test.
 */
void exit_if(int condition, const char *prefix);

/* Write exactly `len` bytes (loops over partial writes). Returns len, or -1
 on error. */
ssize_t write_all(int fd, const void *buf, size_t len);

/* Copy everything from fd_in to fd_out until end of file (read/write loop).
 */
void transfer(int fd_in, int fd_out);

/* Naive recursive Fibonacci: a way to burn CPU time (fibo(40) takes about a
 second). */
unsigned long long fibo(int n);

/*
 * Split argv at the first "--": what comes before goes to args1, what comes
 after to args2.
 * Both arrays end with NULL. Returns -1 if a side is empty or there is no "
 --".
 */
int split_args(char *argv[], char *args1[], char *args2[]);

/* Fork a child that sleeps `seconds` seconds and exits with status 0.
 Returns its pid. */
pid_t create_waiting_child(int seconds);

/* Name of a standard signal ("SIGUSR1"), using the numbering of the running
 system. */
const char *signame(int sig);

/* Print the standard signals that belong to `set` on fd (uses write, async-

```

```
signal-safe). */
void print_sigset(int fd, const sigset_t *set);

/* Print the pending and the blocked signals of the calling process on fd. */
void print_signals(int fd);

/* TD1, exercise 8: the structure written and read in binary by
write_struct/read_struct. */
struct nopad {
    char c1;
    long l;
    char c2;
};

#endif /* FSO_UTILS_H */
```

C tds/td0/errors.c

```
/*
 * TD0 - errors.c: how exit_if() reports a failed system call.
 * open() fails, sets errno, and exit_if prints "open missing.txt: No such
 * file or directory".
 */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "missing.txt";
    int fd = open(path, O_RDONLY);

    exit_if(fd == -1, path);          /* prefix = the file name */
    printf("%s opened as file descriptor %d\n", path, fd);
    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./errors
missing.txt: No such file or directory
[exit status 1]
```

Sortie réelle

```
$ ./errors notes.txt
notes.txt opened as file descriptor 3
```

Quand un appel système échoue, il renvoie `-1` et range le code d'erreur (`ENOENT`, `EACCES` ...) dans la variable globale `errno`. `perror` transforme ce code en texte. C'est pourquoi `exit_if` doit être appelé **juste après** l'appel vérifié: tout appel intermédiaire pourrait modifier `errno`.



Ce qui a été corrigé par rapport à la version héritée

La table des noms de signaux utilisait la numérotation de macOS (sous Linux, `signame(SIGUSR1)` renvoyait un autre nom); `fibonacci` avait des types différents dans `utils.h` et `utils.c`; `create_waiting_child` ignorait son paramètre; et les écritures ne vérifiaient pas leur résultat. La table est désormais construite avec les macros du système et une fonction `write_all` répète les écritures partielles.

TD0 · 4

● Corrigé

Compiler avec `make`

Énoncé. Expliquez les principales règles du `Makefile` du cours: comment `libutils.a` est construite, comment chaque exercice est compilé et comment on obtient plusieurs versions du même programme avec des macros `-D`.

Solution



Texte tds/Makefile

```
# The course compiles with gcc. make's built-in default is CC=cc, which "CC
?= gcc" would not change.
ifeq ($(origin CC),default)
CC := gcc
endif
CFLAGS  ?= -O2 -g
CFLAGS += -std=gnu11 -Wall -Wextra -Werror -D_GNU_SOURCE -pthread -Icommon
LDFLAGS += -pthread
LDLIBS  += -Lbin -lutils -lm

TDS := td0 td1 td2 td3 td4 td5 td6 td7
SRCS := $(foreach t,$(TDS),$(wildcard $(t)/*.c))
BINS := $(patsubst %.c,bin/%,$(SRCS))

# Variants the lab asks to build with different macros (-D...).
VARIANTS := \
    bin/td3/buffer_1 bin/td3/buffer_2 bin/td3/buffer_3 bin/td3/buffer_4
bin/td3/buffer_5 \
    bin/td3/fork_pid_0 bin/td3/fork_pid_1 bin/td3/fork_pid_2 bin/td3/fork_pid_3
\
    bin/td4/mypipe_bug \
    bin/td5/map_none bin/td5/map_read bin/td5/map_write bin/td5/map_exec
bin/td5/map_ew \
    bin/td6/philosophers_deadlock bin/td5/shm_race bin/td6/peterson_fence

.PHONY: all test clean $(TDS) td8

all: bin/libutils.a $(BINS) $(VARIANTS) td8

$(TDS): bin/libutils.a
    @$(MAKE) --no-print-directory $(filter bin/$@/%, $(BINS) $(VARIANTS))

bin/libutils.a: common/utils.c common/utils.h
    @mkdir -p bin
    $(CC) $(CFLAGS) -c common/utils.c -o bin/utils.o
    $(AR) rcs $@ bin/utils.o
```

- `bin/libutils.a` : compile `utils.c` en un objet et l'archive avec `ar`. Un fichier `.a` est une bibliothèque *statique* : l'éditeur de liens ne copie dans chaque exécutable que les fonctions qu'il utilise.
- `bin/%: %.c` est une règle générique: `bin/td1/echo` est construit à partir de `td1/echo.c`. `$<` est le premier prérequis et `$@` la cible.

- `-Lbin -lutils` indique à l'éditeur de liens où chercher la bibliothèque et laquelle utiliser; `-Icommon` indique au compilateur où se trouve `utils.h`.
- `-D_GNU_SOURCE` active toutes les fonctions de la glibc (POSIX et extensions Linux); `-pthread` sera nécessaire pour les threads (TD6).

TD0 · 5

● Corrigé · sortie réelle

Explorez votre système d'exploitation

Énoncé. Trouvez avec des commandes: quel noyau et quelle distribution vous utilisez, quel est le premier processus (PID 1), combien de modules du noyau sont chargés, combien de processeurs vous voyez et combien de temps a duré chaque étape du démarrage.

Solution (sortie réelle sur la machine du cours)

Sortie réelle

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod |
tail -n +2 | wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
237
28
Startup finished in 8.661s (firmware) + 3.568s (loader) + 3.253s (kernel) +
16.571s (userspace) = 32.053s
graphical.target reached after 16.569s in userspace.
```

- `uname` affiche le **noyau** (Linux 7.0); `/etc/os-release`, la **distribution** (Ubuntu 26.04): un même noyau sert à de nombreuses distributions.
- Le PID 1 est `systemd`, le premier processus que le noyau lance au démarrage; tous les autres processus descendent de lui.
- 258 modules chargés: bien que Linux soit monolithique, ses pilotes se chargent et se déchargent à la volée.
- `systemd-analyze` découpe le démarrage en firmware, chargeur, noyau et espace utilisateur: le noyau est l'étape la plus courte. (Il peut être indisponible sous WSL ou dans des conteneurs.)

Ces temps alimentent l'animation A1.3 · Le démarrage →

Liste de vérification

Défi avec l'IA générative · Votre propre Makefile avec des tests

Objectif. Écrivez, avec l'aide d'une IA, un Makefile qui compile chaque exercice d'un laboratoire en son propre binaire et ajoute une cible `test` qui exécute chaque programme avec une entrée connue.

Prompt de départ. Utilisez-le dans l'outil de votre choix (Claude, ChatGPT, Gemini, Copilot, Codex...) avec la version *corrigée* du laboratoire comme contexte. Demandez d'abord un plan et relisez chaque étape avant de l'accepter.

Prompt

Agis comme un tuteur de programmation système en C. J'ai un dossier avec `skeleton.c`, `utils.c` et `utils.h` (bibliothèque `libutils.a`). Je veux un Makefile qui : (1) construit `libutils.a` une seule fois, (2) compile chaque `.c` du dossier en un exécutable avec `-Wall -Wextra -Werror`, (3) a une cible `test` qui exécute chaque programme et compare sa sortie avec un fichier `.expected`. Explique d'abord les règles génériques (%) et les variables automatiques (\$@, \$<, \$^) que tu vas utiliser ; puis donne-moi le Makefile commenté ligne par ligne.

Comment vérifier le résultat

- Lancez `make` deux fois : la seconde fois, rien n'est recompilé.
- Toucher `utils.h` (`touch common/utils.h`) recompile tout ce qui en dépend.
- Cassez volontairement une sortie attendue : `make test` doit échouer.

Déclarez votre usage de l'IA : outil, prompts pertinents, ce que vous avez vérifié vous-même et ce que vous avez corrigé. L'important est que vous puissiez expliquer et vérifier chaque décision.